

Create Agents with AI in CX Agent Studio



Partner

Current points:

star230



1

dashboard

Dashboard

book_2

Catalog

conversion_path

Paths

navigate_next

Gemini Enterprise for Customer Experience: CX Agent Studio Foundationsnavigate_next

Customer Experience Agent Studio: Fundamentalsnavigate_next

Create Agents with AI in CX Agent Studio

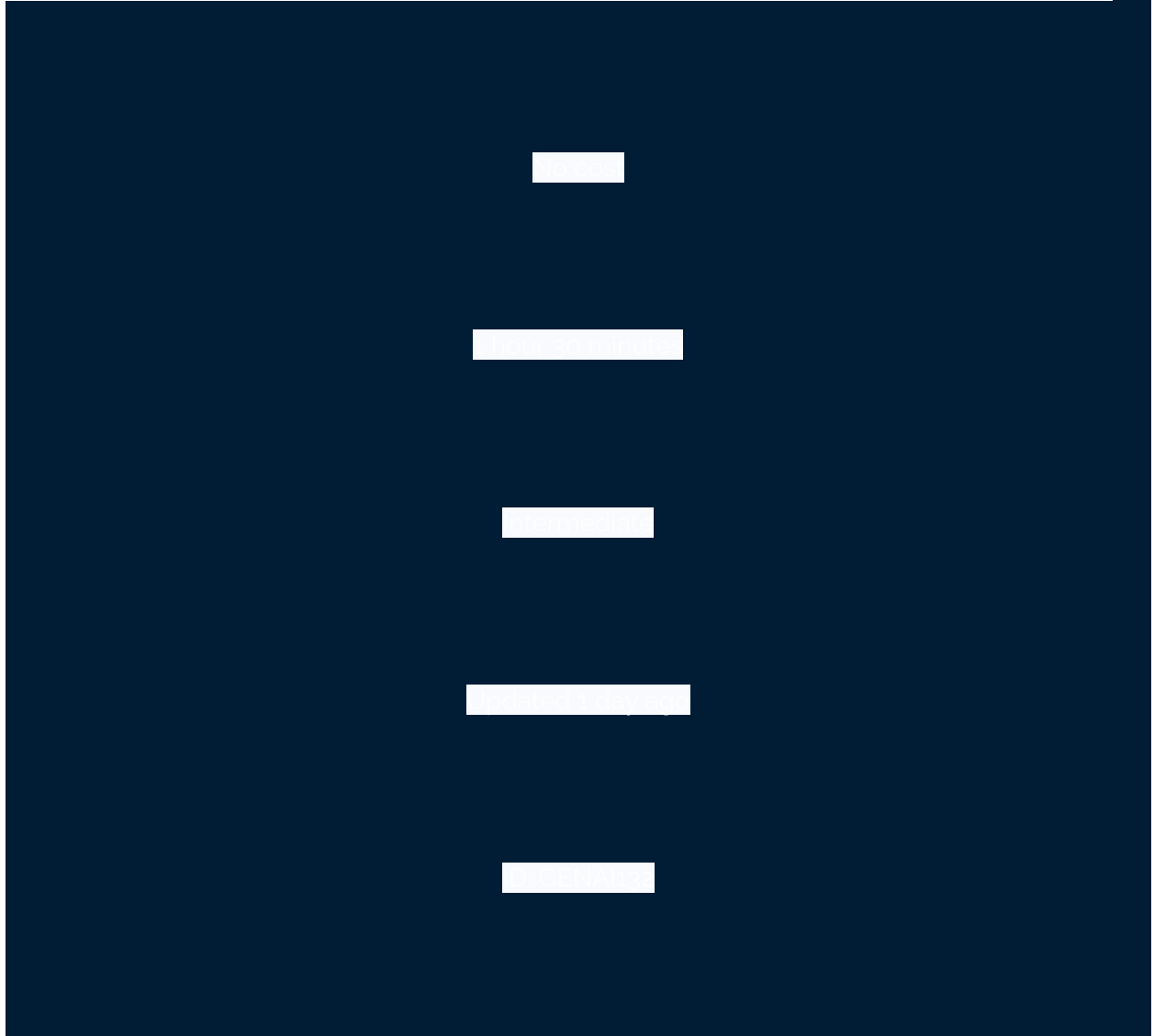
There is a new version of this course available! [Learn more to upgrade to the new version.](#)

Contents

Contents



Create Agents with AI in CX Agent Studio



arrow_drop_down

Score: 0 / 100

timer

1h 8m left

spark

This lab may incorporate AI tools to support your learning.

GENAI132 (gsp1377) / LAB 0

Overview

Customer Experience Agent Studio (CX Agent Studio), a part of the [Gemini Enterprise for Customer Experience](#), helps you create and manage conversational experiences through features like human-like voices, dynamic personalization, multi-modal interactions and intelligent decision making, enabling agents to adapt during conversations.

The new console simplifies agent building with a visual builder interface, allowing both non-technical implementers and developers to create complex CX agents. Additionally, it provides integration with various tools and data sources, such as CRM systems and business messaging platforms, for tasks like transactions, product lookups, and checkout.

Objective

In this lab, you learn how to:

- Build a conversational agent using AI Augmentation and Prebuilt components.
- Explore, test, and reverse-engineer a sample conversational agent.
- Examine agent components such as nodes, instructions, and tools.

- Configure agent elements in the development console.
- Build agents using AI Augmentation by creating a new agent, leveraging Gemini to gather information, and uploading a document to configure the agent.

Setup and requirements

Before you click the Start Lab button

Read these instructions. Labs are timed and you cannot pause them. The timer, which starts when you click **Start Lab**, shows how long Google Cloud resources are made available to you.

This hands-on lab lets you do the lab activities in a real cloud environment, not in a simulation or demo environment. It does so by giving you new, temporary credentials you use to sign in and access Google Cloud for the duration of the lab.

To complete this lab, you need:

- Access to a standard internet browser (Chrome browser recommended).

Note: Use an Incognito (recommended) or private browser window to run this lab. This prevents conflicts between your personal account and the student account, which may cause extra charges incurred to your personal account.

- Time to complete the lab—remember, once you start, you cannot pause a lab.

Note: Use only the student account for this lab. If you use a different Google Cloud account, you may incur charges to that account.

How to start your lab and sign in to the Google Cloud console

1. Click the **Start Lab** button. If you need to pay for the lab, a dialog opens for you to select your payment method. On the right is the Lab setup pane with the following:

- The Open Google Cloud console button
- The temporary credentials that you must use for this lab
- Other information, if needed, to step through this lab

2. In the **Lab Setup** section, right-click **Open Google Cloud console** and select **Open link in new window** (or alternatively right-click and select **Open Link in Incognito Window** if you are running the Chrome browser).

The lab spins up resources, and then opens another tab that shows the Sign in page.

Tip: Arrange the tabs in separate windows, side-by-side.

3. **Note:** If you see the **Choose an account** dialog, click **Use Another Account**.

4. If necessary, copy the **Username** below and paste it into the **Sign in** dialog.

5. `student-03-e22c1f1720ac@qwiklabs.net`

6. Copied!

7. You can also find the Username in the Lab Setup pane.

8. Click **Next**.

9. Copy the **Password** below and paste it into the **Welcome** dialog.

10. `zeCjIo74bq3e`

11. Copied!

12. You can also find the Password in the Lab Setup pane.

13. Click **Next**.

14. **Important:** You must use the credentials the lab provides you. Do not use your Google Cloud account credentials.

15. **Note:** Using your own Google Cloud account for this lab may incur extra charges.

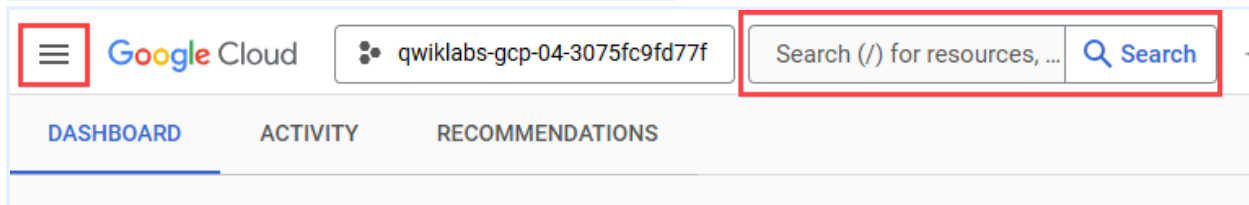
16. Click through the subsequent pages:

- Accept the terms and conditions.

- Do not add recovery options or two-factor authentication (because this is a temporary account).
- Do not sign up for free trials.

After a few moments, the Google Cloud console opens in this tab.

Note: To access Google Cloud products and services, click the **Navigation menu** or type the service or product name in the **Search** field.



Task 1. Set up your Google Cloud project

In this section, you will enable the CX Agent Studio APIs and accept the terms and conditions.

1. Navigate to <https://ces.cloud.google.com/projects>.
2. Choose your project from the list of available projects.

Task 2. Explore a sample agent

In this task you explore, test, and reverse-engineer a sample CX Agent Studio agent to gain practical knowledge and best practices for building high-quality, goal-oriented conversational agents for your business needs.

Your Exploration Checklist

You will examine the agent within the console and complete test scenarios to understand its configuration and live behavior:

- **Agent Nodes:** Analyze the flow and logic of the agent to understand how nodes connect and drive the conversation forward.

- **Instructions:** Review the core instructions that define the agent's persona, purpose, and constraints, and how they shape its responses.
- **Tools:** Investigate integrated tools and their configuration to enable specific actions (e.g., searching a database or accessing external data).

Create the agent

1. Click **Start from an agent template** to start working with it.
2. Select **Plant store demo** from Create agent from template and click **create agent from template**.

You see the creating sample agent pop-up appear.

After a few seconds, the sample agent is created you can **Rename** it to **Cymbal Home & Garden** and is designed to help customers find support with purchasing gardening supplies. Included in the agent are all the associated agent nodes, instructions and tools.

You also see the **Preview agent** panel.



3. Select **cymbal_retail_agent**, click the **three dots**, and then select **Edit config**.

4. Under the **Basic** section, select **gemini-3-flash** from the **Model** dropdown.

The screenshot displays the Google AI Studio interface for configuring an agent. The top navigation bar shows 'Agents > Cymbal Home & ...'. A status bar indicates 'Content Block - Bad Words - First Turn golden evaluation run is complete.' and shows '1 of 10' items, with 'Dismiss' and 'View results' links.

The main workspace is divided into three sections:

- Preview agent:** A chat interface on the left showing a conversation with the agent. It includes a 'Session settings' dropdown and a text input field with a microphone icon.
- Workflow diagram:** A central area showing a flowchart of the agent's logic. It includes nodes for 'cymbal_retail_agent', 'Set as root', 'Create Agent-Tool', and 'Delete'. A red box highlights the 'Edit config' button on the 'cymbal_retail_agent' node.
- Settings panel:** A right-hand panel titled 'cymbal_retail_agent settings'. It contains the following sections:
 - Basic:** Includes 'Agent name' (cymbal_retail_agent) and 'Model' (gemini-3-flash, highlighted with a red box). A note states: 'Selecting a specific model overrides your global settings. This agent node won't update if the global default model is changed.' Below this is a 'Temperature' slider set to 1.
 - Description (optional):** A text area containing 'Retail assistant that helps users with their existing purchases and recommendations for new products based on their needs.'
 - Callbacks:** A section for customizing the agent's behavior with three toggle switches: 'After agent finishes' (checked), 'Before LLM' (checked), and 'After LLM' (checked). A 'Learn more' link is provided.

At the bottom of the settings panel are 'Save' and 'Cancel' buttons.

Copied!

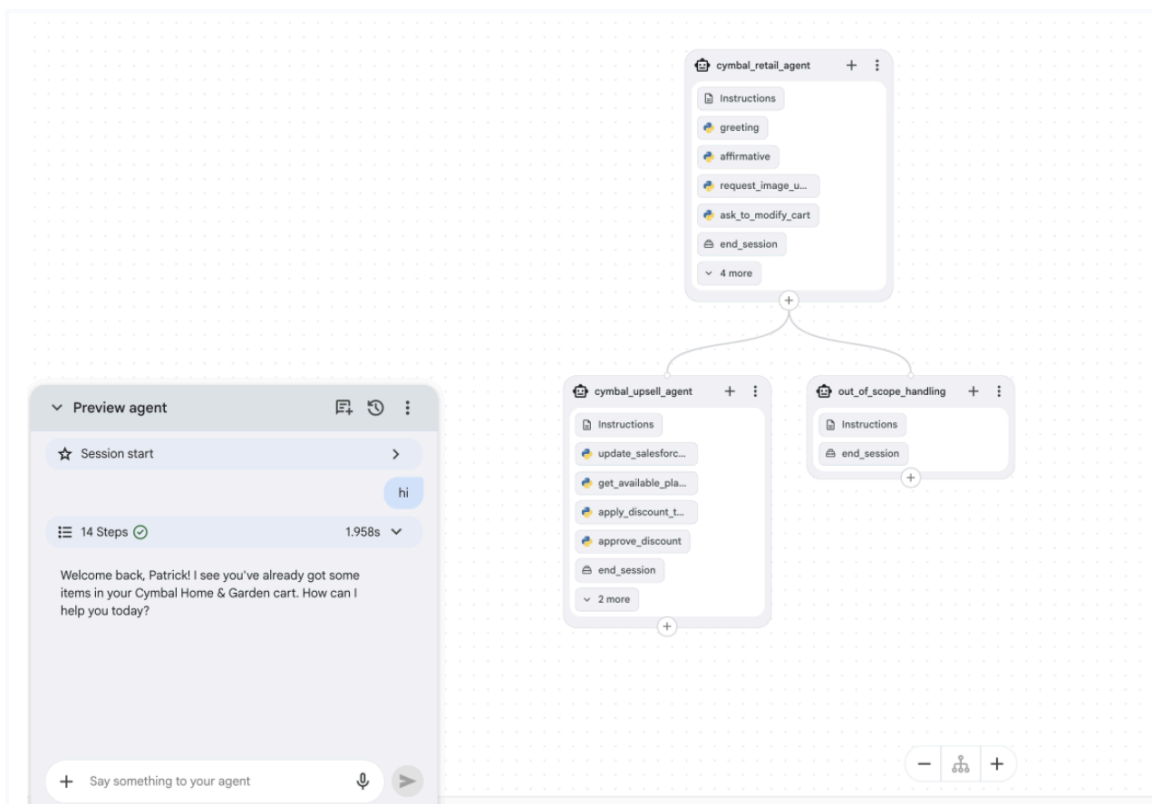
Click **Check my progress** to verify the objective.

Create the agent

Start a conversation and explore tools used for agent responses

1. Enter **hi** and click **send** to start testing the agent.

The agent responds with a greeting and a statement of its purpose.



2. Insert the following text to ask the agent to find care instructions for petunias and click **send**.
3. I'm looking for care instructions for petunias
4. Copied!

5. Notice how each response includes the execution steps that the agent completed to provide a response.
6. Explore the execution steps for this query by expanding the steps completed by the agent to produce the last response. Observe how the `lookup_plant_details` tool is used for the query.

The screenshot displays the 'Preview agent' interface. At the top, it shows '14 Steps' completed in 1.958s. The agent's response is: 'Welcome back, Patrick! I see you've already got some items in your Cymbal Home & Garden cart. How can I help you today?'. Below this, a red box highlights the user's input: 'I'm looking for care instructions for petunias'. The interface then shows another '14 Steps' completed in 4.082s. A button 'Show overview and variables' is visible. The 'Execution steps' section is expanded, showing a timeline of actions: 'Received user message' (0.001s), 'AI reasoning' (1.029s), 'Used tool lookup_plant_details' (1.878s, highlighted with a red box), 'AI reasoning' (2.715s), and 'Agent sent response' (0.001s). A 'Total duration: 4.082s' is shown at the bottom of the execution steps. A button 'Show raw response in JSON' is also present. The final response from the agent is: 'Okay, so for petunias, they really thrive in full sun, meaning about 6 to 8 hours of sunlight every day. They also need well-draining soil and consistent moisture, around 1 to 2 inches of water per'.

7. Expand the **Used tool** section and click on **Tool call** in the execution steps to explore "Input" and "Output" for the tool. You may need to adjust the Preview

agent panel to see all the details.

The screenshot displays the 'Preview agent' interface. On the left, the 'Execution steps' section shows a timeline of actions: 'Received user message' (0.001s), 'AI reasoning' (1.029s), 'Used tool lookup_plant_details' (1.854s), 'After tool call' (0.024s), 'AI reasoning' (2.715s), and 'Agent sent response' (0.001s). The 'Tool call' step is highlighted with a red box. Below the timeline, a 'Show raw response in JSON' button is visible. The main content area shows a conversation with the agent, where the user asks for petunia care instructions. On the right, a 'Used tool lookup_plant_details' panel is open, showing the input and output of the tool call. The input is a JSON object with a 'query' field. The output is a JSON object with an 'answer' field containing detailed care instructions for petunias.

Input

```
1 {
2   "query": "petunias care
3   instructions"
4 }
```

Output

```
1 {
2   "answer": [
3     "Petunias need full sun (6-8
4     hours daily), well-draining
5     soil, and consistent moisture
6     (1-2 inches of water per week).
7     Fertilize regularly, and
8     deadhead spent blooms to
9     encourage continuous flowering.
10    Prune leggy stems to promote
11    bushier growth."
12  ],
13  "search_queries": [
14    "USDA petunias",
15    "petunias care instructions"
16  ]
17 }
```

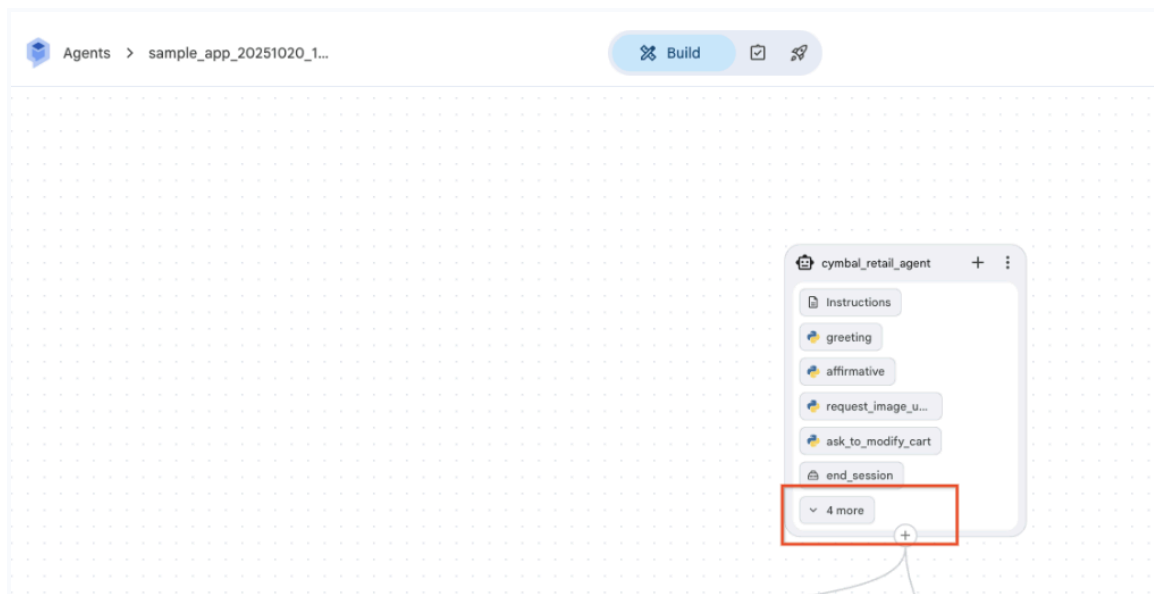
Click **Check my progress** to verify the objective.

Start the conversation

Explore the agent tools and instructions

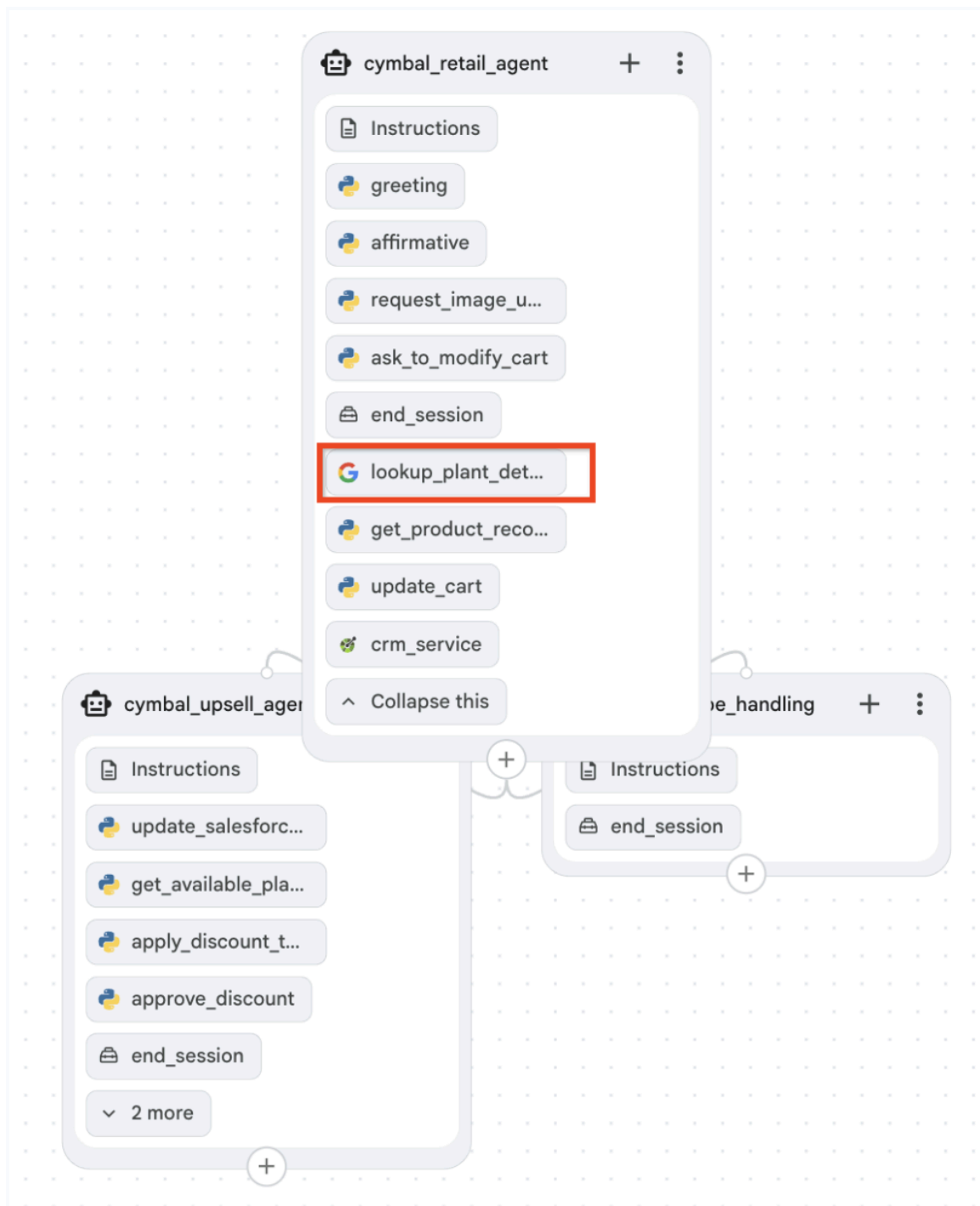
You now explore the agent deeper by learning how each tool and instruction are configured by going through the tool settings in the agent `cymbal_retail_agent`.

1. At the bottom of the `cymbal_retail_agent` root agent, click 4 more to view all tools and instructions included in the root agent.

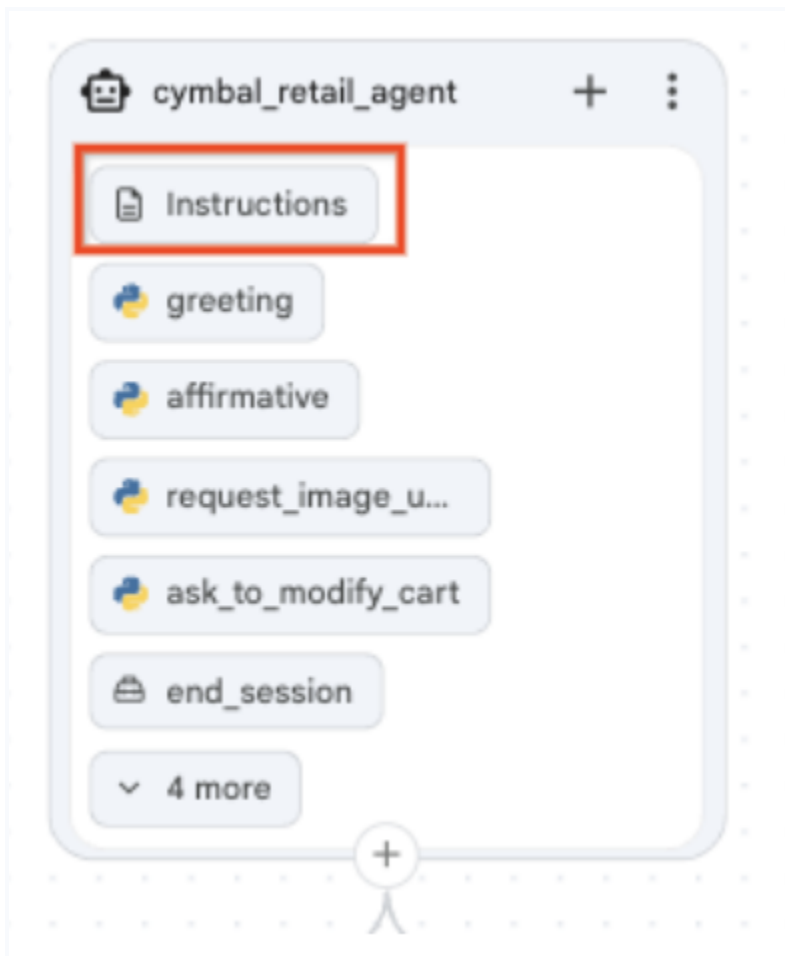


2. Click on the **lookup-plant-details** tool to see tool configuration including the name, description, mock tool response, excluded and included domains as

well URLs for context.



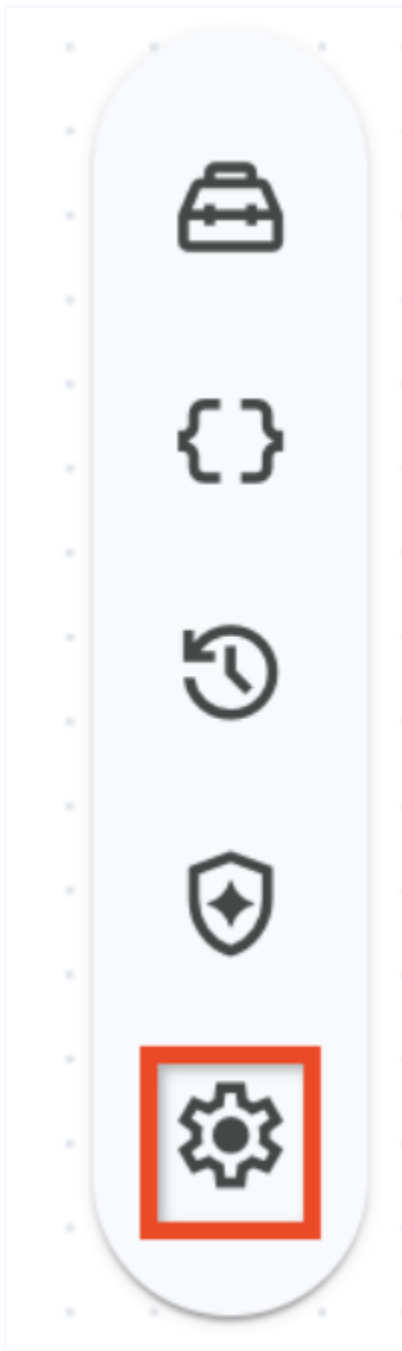
3. Click on **Instructions** to review and understand how instructions are specified for your root agent.



4. Click on other tools and instructions in other agent nodes to explore agent configurations.

Review global settings

1. Navigate to the **Global Settings** menu to review global agent configurations.



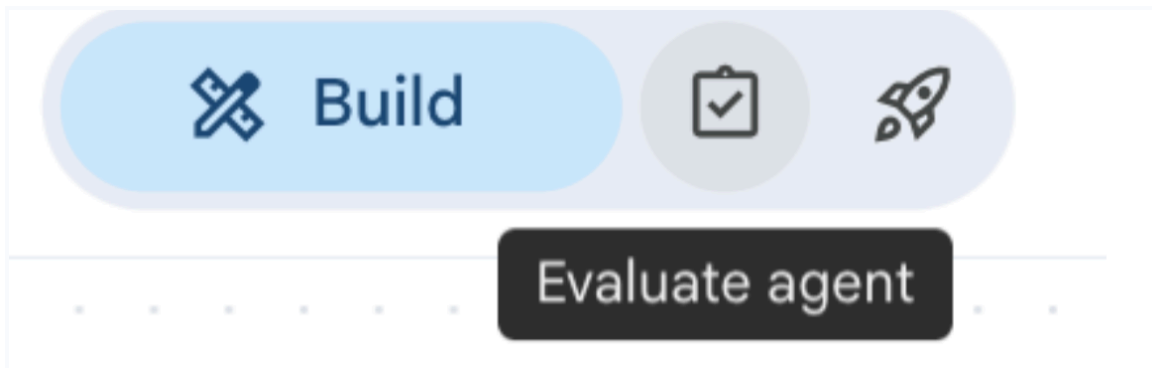
2. You can change the voice of the agent from this menu. Under the Behavior section. Test the default voice **Achnar - Chirp3-HD**, by clicking the **play** button.
3. Use the arrow to change the voice to **Algenib - Chirp3-HD**.

You can explore different voices and test them by interacting with Preview agent using the mic button. You can now make any edits to the agent and tailor it to your custom needs and requirements.

Evaluate the agent

1. At the top of the CX Agent Studio Console, switch from Build to **Evaluate** mode.

The agent comes with a set of prebuilt test cases that can be used to evaluate the agent.



2. Click **Run all** to run tests.
3. Choose a version to run tests against by selecting the **Sample version** and leave the other options as their default value.
4. Click **Run**.

Observe test results and analyse individual conversations to explore metrics:

- Semantic similarity
- Tool correctness

- Hallucination
- Pgo latency

Click **Check my progress** to verify the objective.

Evaluate the agent.

You can now make any edits to the agent and tailor it to your custom needs and requirements.

Task 3. Build agents using AI Augmentation

In this task, you upload an Opportunity Analysis Brief (generated by a Gem in Gemini), so that you can generate a new agent using AI Augmentation.

Create the root agent

1. To get started, navigate to **Agents**.
2. Click **+Create agent**.
3. Name the Agent **Pixel Phone**, keep Enable conversation logging as enabled and click **Create**.

Now you have new empty agent that you will configure and populate with AI generated instructions and tools. The agent will be designed to provide information about the latest pixel phones to new users.

Use AI Augmentation with the brief

As noted, a Gemini Gem was used to create the Opportunity Analysis Brief. See [Gem Build Steps](#) for details.

Within this lab you don't have access to gemini.google.com, so we have provided you steps below to download the PDF for the Brief.

1. Download the [PDF](#) for the Opportunity Analysis Brief to your local computer.
2. Return to CX Agent Studio, with your Root agent displayed and click **Start with AI**.
3. Insert the **goal** (you can find this at the bottom of the Opportunity Analysis Brief).
4. Click **upload** and select the PDF file that you just saved on your computer.
5. Click **Build a preview**.

CX Agent Studio will draft your agent including all the associated instructions and tools.

Give the agent a moment to process the document and generate its instructions, generative playbooks, and necessary tools.

6. Once the draft is ready, click **Create**. This will save the new, fully-configured agent. The process may take a few minutes.
7. Use Preview agent panel to test the agent with chat or voice test, use examples generated by Gemini to test how it performs.

Review the tools and instructions and feel free to customize it.

Click **Check my progress** to verify the objective.

Integrate AI Augmentation into a root agent

Note: The AI Augmentation feature generates sample code for developers to get started. This code is not product-ready and you need to make edits to connect it to your APIs and services to ensure end-to-end execution of tasks. The AI Augmentation functionality doesn't eliminate the need to configure the agent. It's designed to help accelerate the development process, not to fully automate it.

Congratulations!

You have built a CX Agent Studio agent with AI and become familiar with the foundational concepts.

Manual Last Updated June 12, 2026

Lab Last Tested June 12, 2026

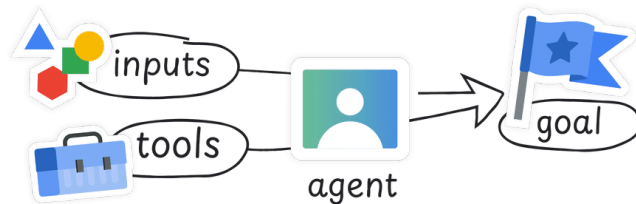
Copyright 2026 Google LLC. All rights reserved. Google and the Google logo are trademarks of Google LLC. All other company and product names may be trademarks of the respective companies with which they are associated.

Skip to lesson

You just learnt about building your own agent, but how does a generative agent work?

Agents

An agent is a piece of software that learns how to best achieve a goal based on inputs and tools available to it.



Agents analyze situations, use multiple tools, and make informed decisions without requiring constant human input. They are also capable of handling multi-step tasks that perhaps an AI model alone cannot, such as researching a topic, troubleshooting code, or accessing a system by chaining together actions. You can have a variety of agents, such as customer agents, code agents, data agents, and many more.

In this course, you'll learn about CX agents. A CX agent is a type of agent that interacts with customers to build relationships and improve their experience with a brand. They handle customer questions, provide support, and try to make the customer feel valued.

Agentic AI

The exact definition of an AI agent can be disputed, so sometimes the term "agentic AI" is used to describe when something has "agentic behavior," meaning it has reasoning and autonomy and is able to complete tasks or accomplish a goal.

Core components of agents

You can think about a generative AI agent as having three core components: Model (brain), tools (abilities), and reasoning loop (instructions or rules).



Foundational models

This is the underlying language model that powers the agent. It could be a small language model or an LLM, a Google model like Gemini, or a model from another provider. The key is to select a model with training data relevant to the agent's intended use case.



Tools

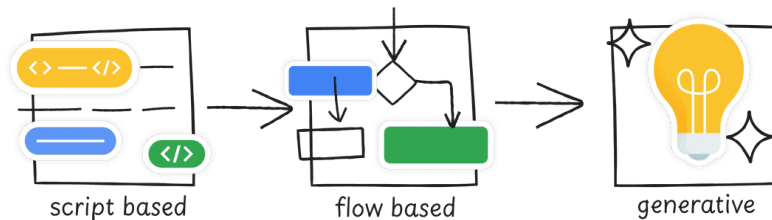
Tools enable the agent to interact with the outside world. These can include extensions that connect to APIs, functions that act as mock API calls, and data stores like vector databases. Tools allow the agent to not only observe the world but also act upon it.



Reasoning loop

This is the core of the agent: Responsible for making decisions and taking actions. It's an iterative process where the agent considers its goal, the available tools, and the information it has gathered. Throughout this cycle, the agent operates within the boundaries of its instructions, using them as a constant reference point to validate decisions and prioritize next steps.

The evolution



Script-based agents (deterministic)

The evolution began with rigid systems reliant on simple keyword detection. These agents followed a strict script. If a user didn't use the exact "magic words" or phrased a question unexpectedly, the agent failed, resulting in a frustrating user experience.

Flow-based agents (deterministic)

Next came agents that utilized manually mapped conversation paths (decision trees) to handle more complex logic. However, they were still limited by the designer's foresight. Every potential branch had to be prebuilt by a developer. If a user went "off-path," the agent hit a dead-end, making these systems difficult to scale and maintain.

Generative agents

The paradigm shifted with the arrival of LLMs that can understand nuance and generate novel responses in real-time without prewritten scripts. Although this unlocks incredible conversational fluidity, it introduces a new challenge: Unlike the previous two methods, you can no longer predict with 100% certainty exactly how the agent will phrase its response.

The new standard

The age of AI is here, and it has completely rewired what your customers expect.

The AI they use every day has set a new, much higher baseline for a good experience, centered on three challenging trends.



Empathetic, multimodal, 24/7 resolution

First, customers now expect multimodal support in near-real-time 24/7. They want to communicate with images and video—not just text.



Seamless blend of AI agents and human support

Second, they expect **intelligent reasoning**. Simple, keyword-based support is obsolete. They expect you to understand the intent of complex questions.



Personalized and proactive engagement

Finally, they demand **hyper-personalization**. One-size-fits-all service is no longer acceptable. Customers expect you to know their history and anticipate their needs. The reason so many businesses struggle to meet this new standard is that they're held back by the challenge of a disconnected and inefficient ecosystem.

The gap

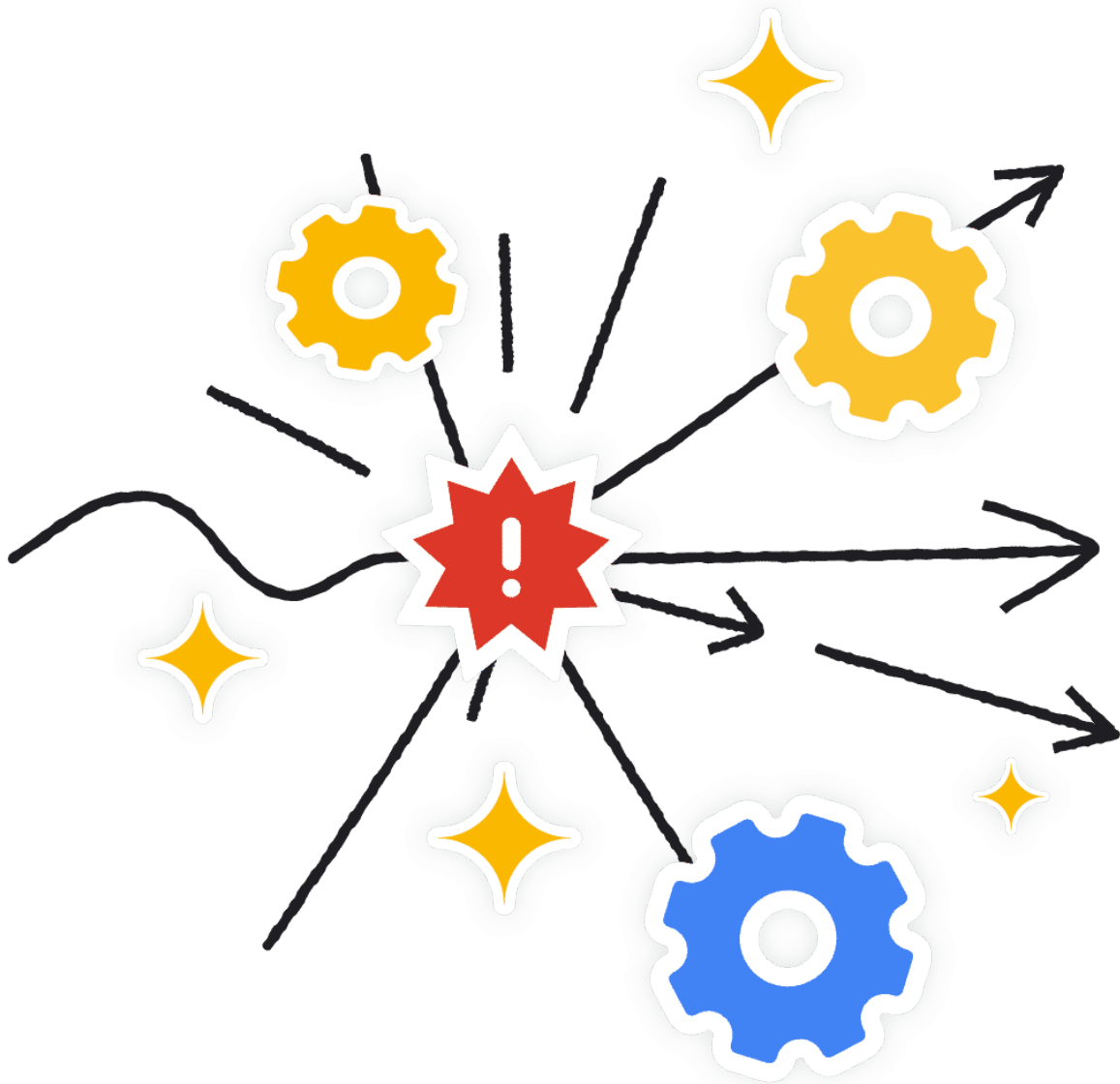
To meet customer expectations, you need to move toward a managed agentic solution to solve today's challenges and future-proof your business.

The reality is that existing capabilities are often mismatched with modern standards.

Complex data systems: Customer data is trapped in fragmented systems, forcing developers to spend time addressing data silos.

Operational stagnation: AI agent development time is spent on iterative prompting and orchestration, complex tool and data integration, and rigorous evaluation data management.

Resource strain: Customer experience virtual agent developers are rare talent, and customer support agents need to provide their expertise for agent design.



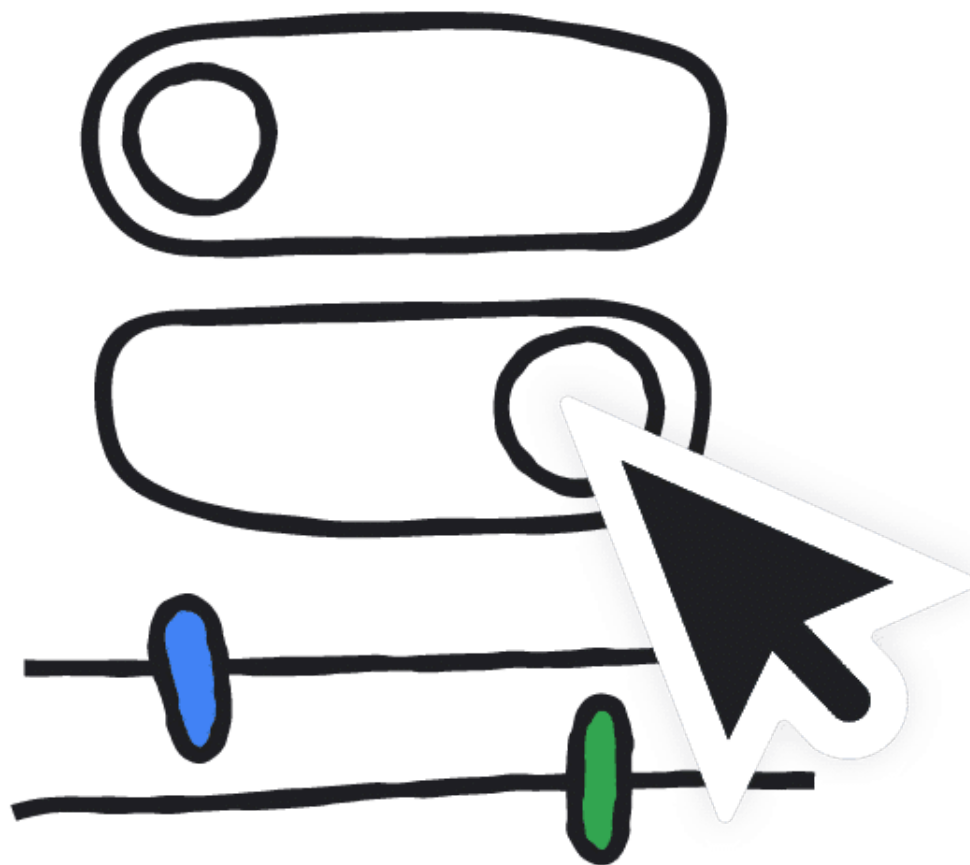
The inflection point

Today, you must move beyond the custom build approach. To solve today's challenges and future-proof your business, you cannot continue shouldering the time, expense, and talent search required to build your own complex infrastructure.

However, there is a paradox: The old scripted methods are too rigid to meet customer expectations, but the new generative methods are too complex and unpredictable to build easily from scratch. The question is no longer *if* you should use AI, but *how* you can harness its power without getting trapped in the plumbing of development.

Skip to lesson

To build a modern agent, you have to leverage and combine two distinct architectures.



Deterministic agents (like the script and flow-based ones) offer 100% control. You know exactly what the agent will say, making them safe for compliance. However, they are rigid. If a user goes "off-script," the agent fails.



Generative agents (AI-based) offer flexibility. They can handle open-ended conversations and "think" on their feet. However, this introduces the "black box" problem: There is a risk of hallucinations (confident lies) or off-brand responses, which creates anxiety for enterprise adoption.

When to use each approach

To build the most effective agent, you need a hybrid agent that uses the right approach at the right time.

Click each tab to learn more about each approach.

Deterministic

Generative

Deterministic execution relies on explicit logic and structured decision trees, making it ideal for high-compliance transactions where precision is paramount and there is zero margin for error. Examples include processing payments, updating account details, or adhering to strict regulatory disclosures.

Example:

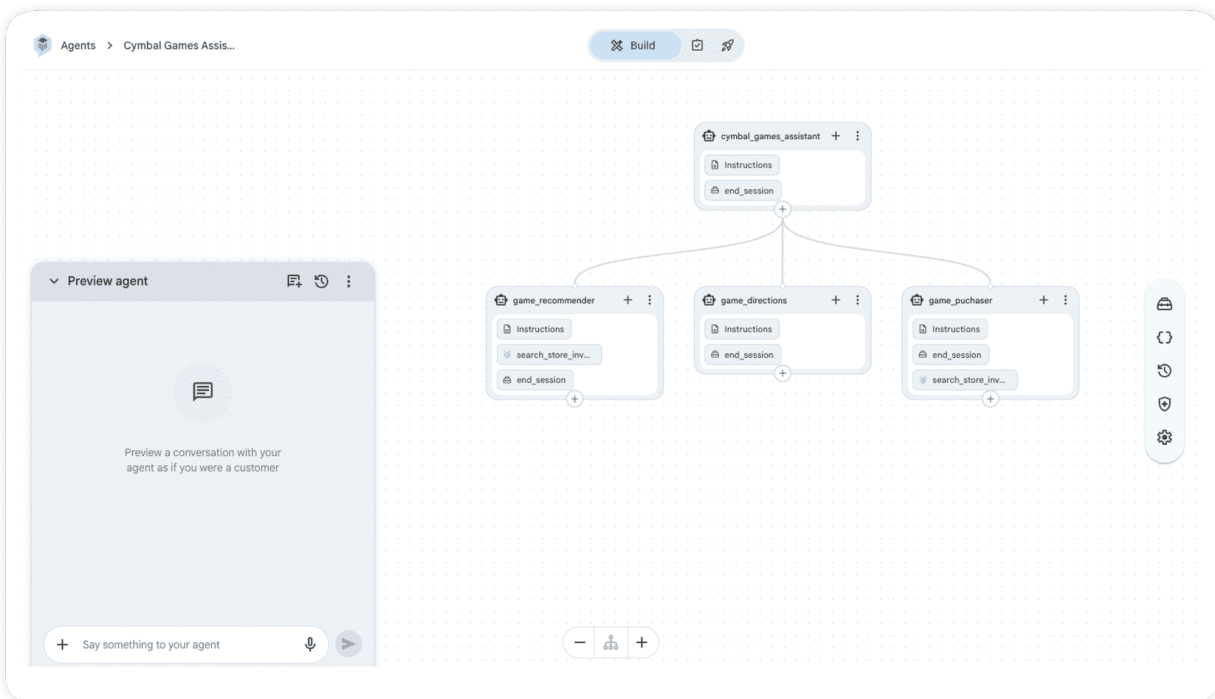
Deterministic	Generative
User: What is my order status? Agent: Your order for the "Espresso Machine" is scheduled for delivery today. User: Thank you!	User: What is my order status? Agent: Great news! I see your order for the "Deluxe Wool Scarf" was delivered to your front porch yesterday afternoon at 4:15 PM. User: I ordered a coffee machine, not a scarf.

This balance in your hybrid agents is critical. The best conversational agents harness the fluidity of generative AI while strictly enforcing the reliability required for business-critical actions.

However, building a system that can switch between these two modes seamlessly is complex. This challenge is exactly what **CX Agent Studio** is designed to solve.

Skip to lesson

You just learned that ideal agents are balanced. They're capable of handling ambiguity and understanding nuance while also enforcing business-critical guardrails and adhering to strict logic. An agent like this needs two different systems (one for language and one for logic), and merging them isn't always seamless. The challenge isn't in generating text or following rules, it's in knowing when to do which because *that* requires a cognitive switch in real-time.



CX Agent Studio is Google's managed, low-code solution designed to solve that exact orchestration challenge. It allows you to build agents that "think" on their feet while

strictly adhering to business logic—like a retail agent that can creatively suggest gifts (generative) while strictly enforcing return policies (deterministic).

Click to zoom the image.

CX Agent Studio is built on the Agent Development Kit (ADK) and operates within the unified Gemini Enterprise ecosystem. The platform is built with an enterprise-grade foundation. It handles the complex infrastructure, allowing you to focus on strategy rather than plumbing.

- **Security:** Built on Google Cloud's secure infrastructure with data encryption in transit and at rest.
- **Integrations:** Out-of-the-box connectors for Salesforce, ServiceNow, and Google Cloud data stores.
- **Debugging:** Advanced simulation tools for voice and text, allowing you to trace the agent's thought process and debug logic visually.

By moving to a managed solution, you stop building the infrastructure and start building the experience.

The core philosophy

Click each tab to learn more.

Generative-first

Multi-agent

Tool-rich

The platform is designed around the power of generative AI as the primary mechanism for logic and creation, making the agent development process dramatically more intuitive and accelerated, primarily through AIA.



Benefits of CX Agent Studio

CX Agent Studio is built for speed and scale.

Expand each of the four items to learn more about CX Agent Studio benefits.

Build instantly

Deploy everywhere

Control quality

Sound natural

Skip to lesson

What is AI augmentation (AIA)

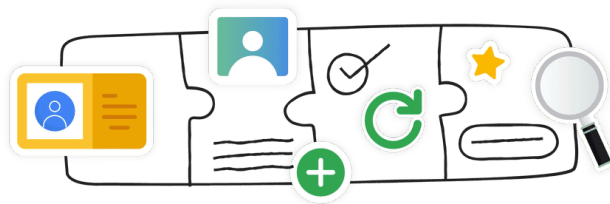
AIA is the ability to use AI to help build your agents. When building CX agents, you have the option to use AIA to accelerate development and improve quality.

[Use AI to help you build AI.](#)

AIA is a Gemini-powered assistant that drastically accelerates how users build, test, and improve conversational agents, enabling faster deployment and higher quality with less manual effort.

Start with AI

You have already seen how you can use **Start with AI** to help you get started creating an agent in CX Agent Studio. Start with AI is just one example of AIA in the product. Later in this course, you will see other use cases for AIA while building your agents.



AI augmentation, not AI automation

Remember, AIA stands for AI augmentation, not AI automation. AIA is there to help you build agents, *not* to take over building your agents. Having humans in the loop in your conversational agent creation is essential. You need to ensure you are always guiding the AI and then reviewing the AI to build the best agent for your customers. AIA should be used to help support you in your initial setup, during brainstorming, in making you more efficient, but it should not replace your experienced eyes and your review. It is the combination of humans with AI that will truly make the best agents to support your customers.

Current points:

star270



1

dashboard

Dashboard

book_2

Catalog

conversion_path

Paths

navigate_next

Gemini Enterprise for Customer Experience: CX Agent Studio Foundationsnavigate_next

Customer Experience Agent Studio: Fundamentalsnavigate_next

Just start designing quiz

There is a new version of this course available! [Learn more to upgrade to the new version.](#)

Contents

Contents

Just start designing quiz

Passing score: 75%

1.

You are a developer tasked with creating a "Travel Concierge" CX agent. You have a clear strategy document outlining the agent's persona and goals, but you have not yet written any code or designed a flow. You need to create a working prototype as quickly as possible to demonstrate the concept to stakeholders. What is the most efficient method to begin this process within CX Agent Studio?

Manually build a decision tree that maps out every possible travel destination.

Use the "Start with AI" feature to describe the CX agent's purpose in natural language, and let the system generate the initial build.

Upload a CSV file of flight schedules to the "Tools" section before configuring the CX agent.

Write a Python script to define the conversation loop, and import it into the console.

2.

A company is upgrading their older "script-based" deterministic chatbot. Customers frequently complained that the old bot would say, "I don't understand," unless they used the exact phrase "pay bill." If they typed "I want to settle my account," the bot failed. What is the primary cause of this failure in script-based CX agents?

They use large language models that are too slow to respond.

They rely on simple keyword detection and lack the ability to understand nuance.

They cannot handle text inputs, only voice.

They suffer from hallucinations and generate confident lies.

3.

A user asks your CX agent, "What is the status of my order #1234?" It understands the question and wants to help, but it replies, "I am unable to look up order information." Which core component is likely missing or unconfigured, preventing the CX agent from retrieving the data?

The Foundational Model

The Persona

The Tools

The Reasoning Loop

4.

You are designing a CX agent for a healthcare provider. It needs to be chatty and empathetic when greeting patients (low risk) but must be precise and predictable when collecting insurance ID numbers to comply with data regulations (high risk). Which architectural approach is necessary for the insurance collection portion of this interaction?

Hybrid

Generative

Probabilistic

Deterministic

All questions must be answered before submitting.

Skip to lesson

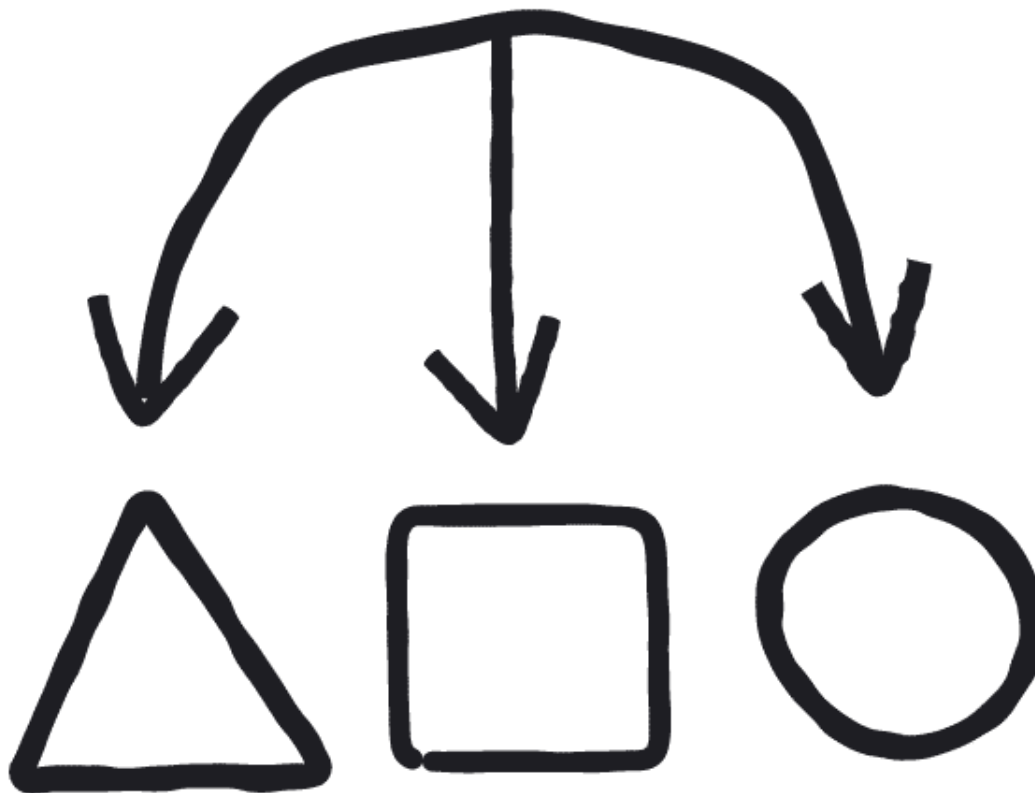
Now that you know how to rapidly prototype an initial agent, let's shift the focus to architecting the specific logic and multi-agent structure. Moving beyond the initial sketch, this is where you'll define your agent's behavior.

*Click the **Play** button to watch the video.*

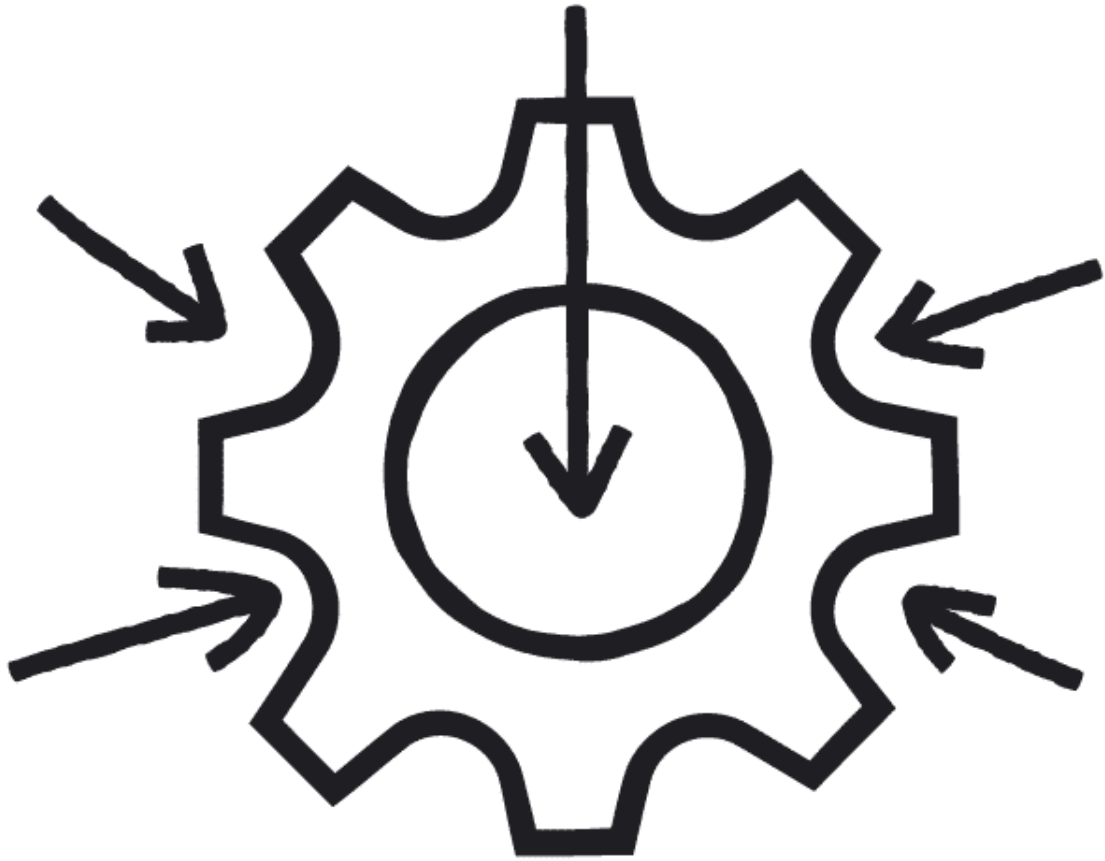
What is a multi-agent architecture

A multi-agent architecture is a design structure that moves away from a single, do-it-all agent and instead organizes your application into a hierarchy of specialized roles.

Think of it as a corporate organizational chart.



The root agent (the manager): This is the main interface that greets the user. Its primary job is delegation. It listens to the user's request, determines which topic it falls under, and routes the conversation to the correct department.



The sub-agents (the specialists): These are agents built to handle one specific domain expertise like "Billing," "Troubleshooting," or "Reservations." They perform the actual work and report back to the manager when finished.

In your specific tool, you can visualize this by thinking of your root sketch as the manager who directs traffic and your sub-agent sketches as the specialists who execute the tasks.

Why use a multi-agent architecture

Expand each of the four items to learn about the benefits of a multi-agent system.

Clarity

Maintainability

Control

Modularity and interchangeability

Collaboration and scale

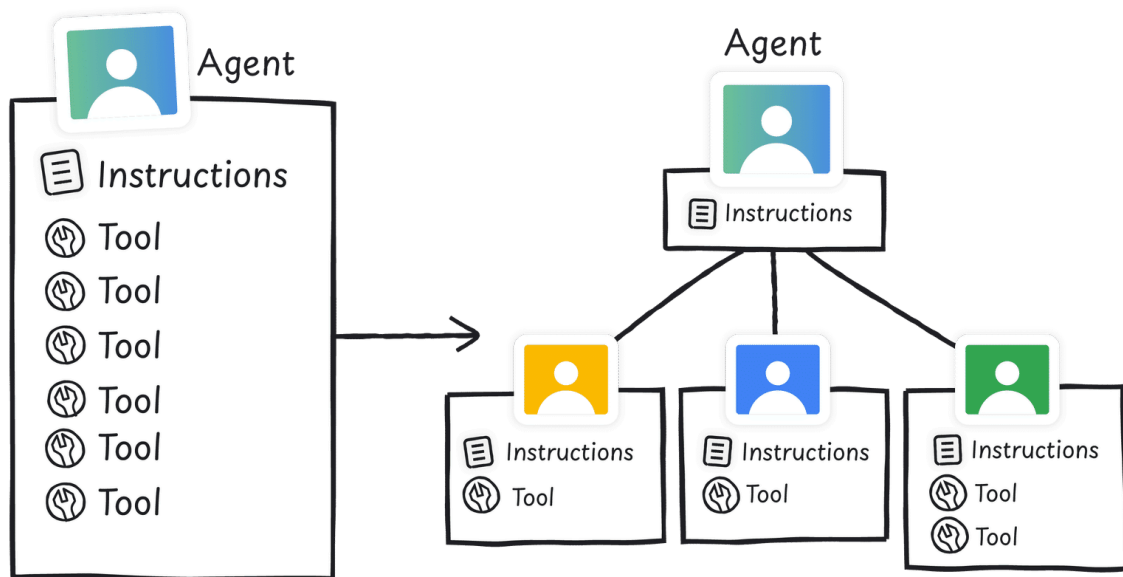
Although these benefits—clarity, maintainability, control, and modularity—are helpful for a solo developer, they become even more essential when a team is working together.

A multi-agent architecture allows different team members to work on different aspects of the project simultaneously. Technically, this means one developer can be building out the logic for a specific sub-agent, while another team member works on a completely different sub-agent at the same time. This separation allows your team to work in parallel, maintain a shared project structure, and scale the application efficiently without blocking each other's work.

When should you divide into multi-agents

How do you decide when to separate a single agent into multiple agents? When should you use a single agent with more instructions and tools versus breaking it into multiple agents?

This is essentially solving the problem: **How do I manage complexity?**



As your agent gets smarter, you'll want it to do more things. As you add this complexity, you have two primary ways to handle the growth: You can either expand that single agent with a longer list of instructions or you can break your single agent into a "team of specialists."

Click to zoom the image.

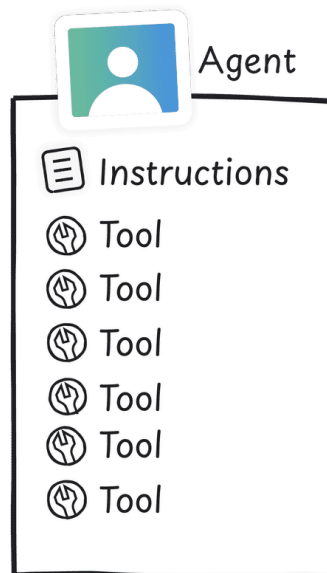
Click each tab to learn about when to break an agent into multiple agents.

Single agent

Multi-agent

In this architecture, you keep a single agent. To make it "smarter," you give it more **Tools** (functions it can call) and add more **Instructions** (text explaining how to use those tools).

- **How it works:** The agent stays in one continuous session. It looks at its list of tools, reads its list of instructions, and picks the right tool for the job.
- **The Risk:** The risk: The "cognitive load" problem. As you add more features, your instructions grow longer. Eventually, the agent may get confused, forget earlier rules, or hallucinate because its context window is too crowded.
- **When to use it:**
 - **The task is "atomic":** It's a quick, one-and-done action (e.g., "Check stock price" or "Convert currency").
 - **The task is stateless:** The agent doesn't need to remember the conversation history. It just needs inputs to give an output.



- If it's a quick "fetch" or calculation, keep it simple: **Single Agent + tool**.
- If it's a complex "process" or conversation, delegate it: **Sub-agent**.

To learn more about this decision process, see the Google Cloud blog on [When to use sub-agents versus agents as tools](#).

[\(opens in a new tab\)](#)